Firmware Guide: how the engine-control software works

Date: 2026-10-11 **Covers:** `firmware/` as merged on master (commit `1149ecc`). **Read with:** decisions log §11 (why it is built this way), §12 (the logs), and the two specs in `docs/superpowers/specs/2026-10-1*`.

This guide explains what the firmware does and how, in the order a new engineer needs it: - what runs where - how one crank tooth and one millisecond are handled - the control loops - safety - CAN - the logs - every tuning knob - how it is tested - what the S32K148 port must provide

---

1. What it is, and where it runs

The firmware is **plain C (C99) that touches no hardware.** It has: - no operating system - no memory allocation at run time - no waiting loops - no chip headers

Because of that, the same files run in three places:

Where	Who calls the firmware	State
A PC	`firmware/sim/run_sil.py`, through a Python model of the engine, generator and fuel rail	**Working: 15 scenarios pass**
NXP S32K148 evaluation board	interrupts and a main loop on the chip, with an Arduino faking the engine signals	Next: the dev-kit spec
Our own ECU board	the same, on our PCB	After layout

It is **bare-metal, not RTOS-based**, on purpose. The work is one 1 ms loop plus a crank-tooth interrupt. A scheduler would add stacks and priority problems without adding anything this ECU needs. The short, fixed overspeed path is also easier to prove without one. If a customer ever requires AUTOSAR OS, the 1 ms entry point becomes a 1 ms task and nothing inside the core changes.

2. Map of the code

File	Job
`core/ecu.h`	The whole interface: the `ecu_io` struct and the entry points. Start here.
`core/ecu.c`	The state machine, the 1 ms schedule, trip handling, the save policy and the watchdog
`core/crank.c`	Reads the 60-2 crank wheel and the cam: synchronisation, speed, angle
`core/inject.c`	Turns fuel and angle into per-cylinder injection commands
`core/govern.c`	Speed governor: PI, setpoint ramp, droop

File	Job
`core/rail.c`	Rail-pressure loop on the metering valve
`core/sensors.c`	Volts to engineering units, range faults
`core/safety.c`	The trip decision and the table of active faults
`core/j1939.c`	Every CAN frame the ECU sends, and request parsing
`core/nvm.c`	The persistent image: layout, checksum, choosing between the two copies
`core/log.c`	Writes into the image: events, counters, fault memory, trip recorder
`sim/test_core.c`	78 unit tests of the C code
`sim/ecu.py`	Python wrapper around the compiled core
`sim/engine.py`	The engine model
`sim/run_sil.py`	The 15 scenarios

All of the core's state lives in `static` variables inside `ecu.c`. There is one ECU per program, and `ecu_init` resets it.

3. The interface

3.1 The `ecu_io` struct

All the firmware sees of the outside world is this one struct. The platform writes the inputs and reads the outputs.

Group	Fields	Meaning
Inputs	`v_clt`, `v_oil`, `v_rail`, `v_boost`, `v_egr`	Sensor voltages at the ADC pins (V)
	`v_5v`	Measured 5 V sensor supply; the ratiometric sensors are corrected by it
	`v_bat`	Battery voltage, recorded in fault snapshots
	`ignition`, `droop_sw`	Switch inputs, 0 or 1
	`reset_cause`	Set once before loading: 0 power-on, 1 watchdog, 2 other
Outputs	`inj[3]`	Per cylinder: start time, duration (µs) and a sequence number `seq`
	`meter_duty`	Metering-valve duty, 0 to 1
	`wdg`	The watchdog pin level
	`can[16]`, `can_n`, `can_drops`	CAN frames to send; the platform sends `can_n` of them, then zeroes it
	`nvm_save`, `nvm_slot`	"Write the image to this copy now", cleared by the platform when done
Status	`state`, `rpm`, `fuel_mg`, `rail_bar`, `clt_c`, `oil_kpa`, `fault_n`, `synced`	For display and tests

Injection is given as commands, not pin levels. For each cylinder the command is "start at time T, for D microseconds". The chip's timer output-compare runs that directly. Every change to a command,

including a cancel (duration 0), increments `seq`, so the platform only needs to watch `seq`. A cancel stops a command that hasn't started. An injection already open runs to its end.

3.2 Entry points

Function	Called by	How often	Does
`ecu_init`	start-up	once	Resets everything
`ecu_nvm_load`	start-up, after init	once	Loads the newer good copy of the image, logs the power-up
`ecu_crank_edge`	crank-sensor interrupt	every tooth (about 1450/s at 1500 rpm)	Decodes the tooth, **checks overspeed**, schedules injection
`ecu_cam_edge`	cam-sensor interrupt	once per 2 revolutions	Marks which revolution has cylinder 1 firing
`ecu_tick_1ms`	1 ms timer interrupt	1000/s	Everything slow (§5)
`ecu_can_rx`	CAN receive	per frame	Answers J1939 requests (§9)
`ecu_background`	main loop	as often as it likes	Saves, and checks in for the watchdog
`ecu_nvm_image`	main loop	when `nvm_save` is set	Gives the sealed image to write

One rule for the port: `ecu_crank_edge`, `ecu_cam_edge`, `ecu_tick_1ms` and `ecu_can_rx` share data and must not interrupt each other. Give their interrupts the same priority. `ecu_background` runs at the lowest level and may be interrupted at any time.

4. One crank tooth

This is the fastest and most important path. At 1500 rpm a tooth arrives every 0.67 ms.

1. **Noise filter.** A tooth that arrives in under half the previous tooth's time is ignored. No engine accelerates that fast, so it can only be noise. A tooth arriving after more than 50 ms of silence restarts the decoder's history.
2. **Gap and sync.** The wheel has 58 teeth and a gap where two are missing, so the gap shows up as a period more than 1.5× the last one. Two gaps exactly 58 teeth apart mean **synchronised**. Once synchronised, a gap in the wrong place, or no gap where one belongs, is **sync loss**.
3. **Phase.** The crank turns twice per engine cycle, and the cam pulse says which turn has cylinder 1 firing. Until a cam pulse has been seen, no fuel is injected. A cam pulse in the wrong turn stops injection until the next one agrees. It does not shut the engine down, because one noise spike shouldn't stop a genset.
4. **Speed.** rpm comes from the last 6 tooth periods, with the gap counted as its true 18°.

5. **Safety, on this very tooth.** If speed is above **1650 rpm**, the ECU shuts down here, inside the interrupt, and cancels every injection that hasn't started. If this tooth itself raised a sync fault (a lost tooth reads as a speed jump), it is diagnosed as a crank sensor fault, not overspeed.
6. **Injection.** Each cylinder starts injecting **5° before top dead centre** (cylinder 1's TDC is 90° after the gap; cylinders 2 and 3 follow at 240° and 480°). The command is written on the last tooth at least **1° before** that angle, so the start time is never already past. The start time is extrapolated from the current tooth speed.

5. One millisecond

`ecu_tick_1ms` runs, in this order:

1. The ms counter, which wraps every hour (3 600 000) by design.
2. **Watchdog:** toggle the WDI pin every 5 ms (a falling edge every 10 ms), but **only if the main loop has checked in within 50 ms**.
3. Once a second: the counters (§10).
4. Ignition handling: the shutdown latch (§6), and the ignition-off event after 500 ms off.
5. Crank timeout check: 50 ms without a tooth while synchronised is sync loss.
6. Sensors: convert, range-check, update the fault table (§8).
7. Safety check (overspeed, sync loss, ignition off).
8. Governor (in RUN) and rail loop (in CRANK and RUN).
9. CAN messages due this millisecond (§9).
10. A trip-recorder sample every 20 ms.
11. The status fields.

6. The state machine

```
OFF ──ignition on──▶ WAIT ──first tooth──▶ CRANK ──phased and ≥ 400 rpm──▶ RUN
                                                    (any state)
            overspeed > 1650 rpm  ─┐
            sync loss while RUN   ─┼──▶ SHUTDOWN (latched)
            ignition off          ─┘
SHUTDOWN ──ignition off ≥ 500 ms with the engine stopped, then on──▶ OFF ──▶ WAIT
```

- **WAIT:** powered, ignition on, waiting for the engine to turn.
- **CRANK:**
- the starter is turning the engine
- fixed start fuel of 30 mg once phased
- a decoder fault just restarts synchronisation
- **RUN:**
- the governor owns the fuel

- on entry the speed setpoint starts at the current speed and ramps at 500 rpm/s to 1500, which stops the start from overshooting towards the trip
- **SHUTDOWN:**
- no injection
- metering valve closed
- fault codes active
- CAN keeps reporting, so the DSE sees the engine run down

The latch clears only on a real ignition cycle: ignition off for at least 500 ms *with the engine stopped*, then on. A bouncing contact or a flicker while the engine still turns does nothing.

7. The control loops

Governor (`govern.c`): - PI on the speed error, output = fuel per injection, clamped to 0–45 mg. - Isochronous at 1500 rpm. - With the droop switch closed, the setpoint falls 4 % from no load to full load (34 mg, about 20 kW). - The integrator is clamped (anti-windup). - Gains: 0.5 mg/rpm and 2.0 mg/(rpm·s), tuned in simulation, which gives a 3.4 % dip (1500 to 1449 rpm) on a full 20 kW step.

Rail pressure (`rail.c`): - PI drives the metering valve to hold **1000 bar**. - The valve is assumed **normally closed**: no power, no fuel; more duty, more flow. This matches the trip module, which stops fuel by cutting the valve's supply. **The real valve is not yet identified.** If it turns out to be normally open, `METER_NC` flips *and* the trip module's wiring has to move. - In SHUTDOWN the valve is commanded closed. - If the rail sensor fails, the duty is fixed at 0.3 so the engine keeps running.

Injection duration (`inject.c`): duration = 300 µs dead time + fuel ÷ (0.0009 × √rail pressure). Rail pressure is floored at 100 bar, so a failed or zero reading can't produce an enormous pulse.

8. Sensors

Every input is corrected by the measured 5 V supply. If that supply reads outside 4.5–5.5 V, 5.0 V is used instead.

Sensor	Scaling (placeholder)	Valid volts	Default on fault	J1939 SPN
Coolant temperature	NTC 2.5 kΩ at 20 °C, β 3450, 2.49 kΩ pull-up	0.1–4.8 V	90 °C	110
Oil pressure	0.5–4.5 V = 0–1000 kPa	0.2–4.8 V	300 kPa	100
Rail pressure	0.5–4.5 V = 0–1800 bar	0.2–4.8 V	1000 bar	157
Boost pressure	0.5–4.5 V = 50–400 kPa	0.2–4.8 V	100 kPa	102
EGR position	0.5–4.5 V = 0–100 %	0.2–4.8 V	0 %	27

Range faults: - Out of range for **100 ms** raises a fault: FMI 3 for too high (an open circuit), FMI 4 for too low (a short). - **From the first moment out of range, the last good value is held.** The 100 ms delay only delays the fault code; it never lets a bad value through. - While a channel is faulted, its default is used for control and "not available" (0xFF) is sent on CAN. - The coolant low limit is 0.1 V, not 0.2 V. 0.2 V is a real 129 °C, and a hot engine must not be mistaken for a shorted sensor.

9. CAN (J1939, source address 234)

Sent all the time:

Message	PGN	Every	Carries
EEC1	61444	20 ms	Engine speed (0.125 rpm/bit)
EFL/P1	65263	500 ms	Oil pressure
ET1	65262	1 s	Coolant temperature
DM1	65226	1 s	Active faults and lamps (red = shutdown fault, amber = sensor fault)
HOURS	65253	10 s	Engine hours (0.05 h/bit)

Sent on request (a J1939 request addressed to 234 or to everyone):

Request	PGN	Answer
DM2	65227	Stored (previously active) faults
DM3	65228	Clears stored faults; active faults are untouched
DM4	65229	Fault snapshots: speed, load, coolant, then rail, oil, battery, state
HOURS	65253	Engine hours

Long messages. A message longer than 8 bytes goes as a broadcast sequence: one announcement, then packets 50 ms apart. Three rules protect it: - Only one sequence is in flight at a time. - The active fault list (DM1) always goes before DM2/DM4, so a service tool can't starve the controller of faults. - A sequence is sent whole or not at all: no announcement, no packets, and a packet that meets a full queue is retried 1 ms later.

The exact set of messages the DSE4522 expects is still open item U4.

10. The logs

Everything below lives in one **persistent image** of 3036 bytes, which fits the S32K148's 4 KB emulated EEPROM.

Record	Holds
Counters	ECU powered seconds, engine hours, starts, trips by cause, watchdog resets, time at each quarter of rated load
Fault memory	16 faults; each with an occurrence count, first and last time seen, and a **snapshot taken the first time**: speed, load, fuel, rail pressure, coolant, oil, battery, state

Record	Holds
Event log	The last 64: power-up (with cause), ignition on/off, crank, run, shutdown (with reason), fault set/cleared, stored-fault clear, image reset
Trip recorder	The last 2 overspeed or crank-signal trips: speed, fuel, rail pressure and state every 20 ms, from 2 s before to 1 s after

Saving: - The platform keeps **two copies** and writes them alternately. Each copy carries a sequence number and a CRC-32 over the whole image. At power-up the newer good copy wins, so a power cut during a write costs one save, never the history. - An image from a different layout version is never misread. It is treated as blank, and an "image reset" event is logged. - At power-up, every stored fault is marked previously active until it is seen again. - Saves are requested only by `ecu_background`, never from an interrupt, on any of these: - a fault's first occurrence - each start - entering shutdown - each hour of running - a watchdog-reset boot - a trip (1 s after it, once the recording is complete) - Saves are capped at one per 10 s, except a trip. That is about 30 a day, roughly 110 000 per copy over 20 years. - A fault's set and clear are logged once per ignition cycle, so a loose connector can't flood the event log.

Reading: stored faults, snapshots and engine hours over J1939 (§9). The event log and trip recorder need the laptop service tool from the dev-kit spec. Until then, the simulator decodes them from the image (`decode()` in `run_sil.py`).

11. Safety, in layers

Layer	Catches	Where
Overspeed check on every tooth	Runaway: trips at 1650 rpm, under the DSE's own 1680/1710 rpm trips	`ecu_crank_edge`
Sync-loss trip	Lost crank signal (50 ms) or a wheel that no longer makes sense	`crank.c` , `ecu.c`
Injection cancel	Any command not yet started when a trip happens	`inject_cancel`
Metering valve closed in SHUTDOWN	Rail pressure falls after a trip	`rail.h METER_CLOSED`
Window watchdog (TPS3850)	A hung main loop: the pulses stop and the chip resets the processor; the next power-up logs it	`ecu_tick_1ms` , `ecu_background`
Independent trip module	Everything above failing: it cuts the metering valve's supply from outside the board	Hardware, spec §3

The firmware is the first line, not the only one. The trip module doesn't depend on the firmware at all.

12. Every tuning knob

Each is one `#define`. *Assumed* means a guess to be replaced with the real engine's figure. *Calibration* means tuned in simulation and to be re-tuned on the engine.

Knob	File	Value	Status
`CRANK_TDC1_DEG`	crank.h	90° after the gap	assumed (60-2 wheel assumed; **firing order 1-2-3 confirmed** 13 Oct 2026)
`CRANK_TIMEOUT_US`	crank.h	50 ms	design
`OVERSPEED_RPM`	safety.h	1650	design
`GOV_RPM_SET`, `GOV_DROOP`	govern.h	1500, 4 %	design
`GOV_KP`, `GOV_KI`	govern.h	0.5 mg/rpm, 2.0 mg/(rpm·s)	calibration
`GOV_RAMP_RPM_S`	govern.h	500 rpm/s	calibration
`FUEL_MAX_MG`, `FUEL_RATED_MG`	govern.h	45, 34 mg	assumed
`FUEL_START_MG`	inject.h	30 mg	calibration
`INJ_SOI_BTDC`	inject.h	5°	calibration
`INJ_DEAD_US`, `INJ_K`	inject.h	300 µs, 0.0009	assumed (injector data)
`METER_NC`	rail.h	1 (normally closed)	**assumed: identify the valve**
`RAIL_SET_BAR`	rail.h	1000 bar	placeholder (real maps vary it with speed and load)
`RAIL_KP`, `RAIL_KI`	rail.h	0.002, 0.01	calibration
`RAIL_DUTY_LIMP`	rail.h	0.3	calibration
Sensor tables	sensors.c	§8	assumed until each front-end is measured
`RUN_RPM`	ecu.c	400	design
`IGN_OFF_MS`	ecu.h	500 ms	design
`SAVE_GAP_S`	ecu.h	10 s	design (flash wear)

The engine model in `sim/engine.py` has its own assumed constants, also marked. When real engine data arrives, change the constant, re-run the scenarios, and they say at once whether the control still holds.

13. How it is tested

```
make -C firmware/sim test                       # 78 unit tests, about 1 s
.venv/bin/python firmware/sim/run_sil.py        # 15 scenarios, about 2-3 min
.venv/bin/python firmware/sim/run_sil.py rail   # scenarios whose name contains "rail"
```

#	Scenario	Proves
1	Start	in sync within 2 revolutions; 1500 rpm by 5 s; peak under 1600
2	Load step 0 → 20 kW	dip stays above 1350 rpm; back within ±15 rpm in 5 s
3	Load rejection 20 → 0 kW	peak stays under the trip

#	Scenario	Proves
4	Droop switch	speed falls 4 ± 1 % at full load
5	Stuck injector (runaway)	shuts down on the very tooth it first reads > 1650 rpm; no injection after; rail pressure falls
6	Crank signal lost	shutdown within 100 ms, crank-sensor fault
7	Ignition off	shutdown with no fault code
8	Coolant sensor unplugged	fault within 1.2 s; engine keeps running
9	CAN timing and content	every message on time; speed within 0.5 %
10	Rail pressure	within 5 % of 1000 bar
11	Watchdog	pulses every 10 ms, right to the end of the run
12	Runaway, then power cycle	fault, snapshot, event and trip recording all survive
13	Power cut mid-save	the previous save loads
14	Stored faults over CAN	DM2 reads them; DM3 clears them and leaves active ones
15	One lost crank tooth	logged as a crank-sensor fault, not overspeed

What a pass means, and what it doesn't. The scenarios prove the logic and the safety behaviour. They don't prove the tuning on a real engine. The engine model uses the firmware's own injector and sensor formulas, and it has no electrical noise. Two independent expert reviews, each reproducing its findings, were run against the code (decisions log §11 and §12).

14. What the S32K148 port must provide

The core needs nothing else. The platform layer must:

1. **Crank input:** input capture on the crank sensor; call `ecu_crank_edge` with the capture time in µs (a free-running 32-bit counter). Same for the cam.
2. **1 ms timer:** call `ecu_tick_1ms` with the same µs clock.
3. **Injection:** after each call, if `inj[k].seq` changed, load `start_us` / `dur_us` into that cylinder's output-compare channels. Duration 0 means disarm, unless the injection has already started.
4. **Metering valve:** drive the PWM from `meter_duty`.
5. **CAN:** send `can[0..can_n)` and zero `can_n`. Pass received frames to `ecu_can_rx`.
6. **Watchdog:** copy `wdg` to the TPS3850's WDI pin. Strap its window (SET0/SET1, CWD) to accept a 5–20 ms kick period.
7. **Storage:** keep two copies of the image. When `nvm_save` is set, write `ecu_nvm_image()` to copy `nvm_slot`, **without blocking the main loop for more than 50 ms** (or the watchdog resets the chip), then clear

`nvm_save`. At start-up pass both copies to `ecu_nvm_load`, with `reset_cause` from the chip's reset status.

8. **Main loop:** call `ecu_background` continuously.
9. **Priorities:** the edge, tick and CAN-receive interrupts at one priority. Mask them during the image copy inside `ecu_background`.

15. Known limits and open items

Needs the machine or a supplier: - the real metering valve - crank wheel and firing order - the sensor tables - injector flow data

Needs the real engine: - governor and rail tuning - start fuel - injection timing maps - pilot and post injections

Needs U4 (the DSE's exact expectations): - the exact J1939 message set - whether DM3 needs an acknowledgement

Dev-kit spec: - flash choice for about 110 000 writes per copy - the laptop service tool (event log, trip recorder, calibration) - the bootloader for field updates - MISRA C checks - governor performance graded against ISO 8528-5 - a failure analysis of the overspeed path

Known minor items: - An extra crank tooth from a noise spike still logs as overspeed. The engine stops correctly either way. - A faulted sensor's snapshot stores its default value rather than "not available". - DM1 always reports an occurrence count of 1.